

1. DNN 집값 예측하기

1) 실습문제 1/7

**MY_EPOCH 하이퍼 파라미터를
0으로 바꾸고 실험을 반복하세요.**

결과가 어떻게 바뀌나요?

1) 실습문제 1/7

(1) 문제 분석

- MY_EPOCH는 기계 학습 시 학습 데이터를 몇 번을 반복하며 학습할 지를 지정
- 0으로 세팅한다는 것은 학습을 한 번도 안 한다는 의도
- 가중치와 편향값을 보정하지 않고 임의의 초기값 그대로 사용한다는 의미

(2) 예상 답

- 손실 값: MSE 증가 예상
- 학습 시간: 0으로 예상

**MY_BATCH 하이퍼 파라미터를
16으로 바꾸고 실험을 반복하세요.**

결과가 어떻게 바뀌나요?

(1) 문제분석

- MY_BATCH는 학습 데이터를 매번 메모리에서 몇 개씩 가져와서 계산할 지를 지정
- 우리가 사용하는 학습 데이터에는 354개의 집값 데이터가 있음
- 매번 64개씩 가져와서 병렬계산 하지 않고 16개씩 가져와서 계산하면 학습시간이 상당히 증가함

(2) 예상 답

- 정확도: 영향 없음 예상
- 학습 시간: 상당한 증가 예상

2) 실습문제 2/7

DNN 학습 최적화

```

1 # TensorFlow (텐서플로) 환경 구축하기
2 # 2028년 8월 28일
3 # 김성규 교수, 미국 조지아 공대
4 # liask@ece.gatech.edu
5 # 한국기술교육대학교 온라인 평생교육원
6
7 # 파이썬 패키지 수입
8 import pandas as pd
9 import matplotlib.pyplot as plt
10 import seaborn as sns
11 from time import time
12
13
14 from keras.models import Sequential
15 from keras.layers import Dense
16 from sklearn.preprocessing import StandardScaler
17 from sklearn.model_selection import train_test_split
18
19
20 # 하이퍼 파라미터
21 MY_EPOCH = 500
22 MY_BATCH = 64
  
```

DNN 요약
Model: "sequential_1"

Layer (type)	Output Shape	Params
dense_1 (Dense)	(None, 200)	2600
dense_2 (Dense)	(None, 1000)	20100
dense_3 (Dense)	(None, 1)	1001

Total params: 204,601
Trainable params: 204,601
Non-trainable params: 0

DNN 학습 시작
총 학습 시간: 7.8초
DNN 평균 제곱 오차 (MSE): 0.19

실습문제 2

◆ 실습문제 2

- MY_BATCH 하이퍼 파라미터 64 → 16로 변경
- 주의할 점: MY_EPOCH 0 → 500으로 원상복구
- 평균제곱오차: 0.19 / 총 학습시간: 7.8
- MY_BATCH 하이퍼 파라미터 16 → 64로 변경
- 평균제곱오차: 차이 없음 / 총 학습시간: 20

<정답>

- 손실 값: 영향 없음
- 학습 시간: 7.8초에서 20.3초로 약 3배 증가

<추가 문제>

- Batch 크기를 354로 늘려보세요.

1) 실습문제 3/7

500개 뉴런을 가진 은닉층 두 개를

마지막 단에 추가하고 실험을 반복하세요.

결과가 어떻게 바뀌나요?

1) 실습문제 3/7

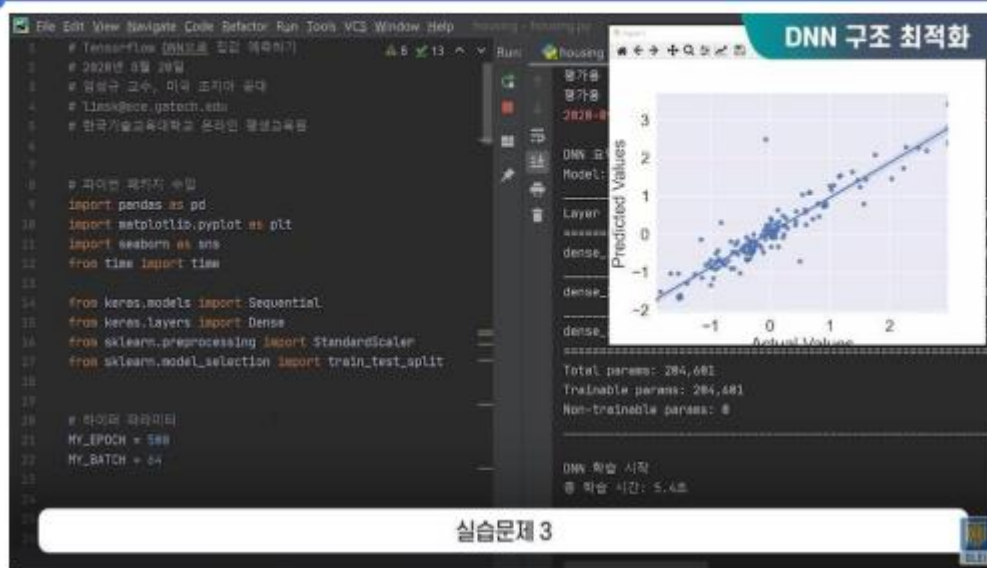
(1) 문제분석

- 현재 DNN은 두 개의 은닉층을 가짐: 200개 뉴런 한층, 1,000개 뉴런 한층
- 여기에 500개짜리 두 개를 추가함
- 심층신경망에서 은닉층의 숫자가 많을수록 손실 값은 감소함
- 보정해야 할 가중치의 수도 늘어나, 학습시간이 증가함

(2) 예상 답

- 손실 값: MSE 개선 예상
- 학습 시간: 증가 예상

1) 실습문제 3/7



◆ 실습문제 3

- MY_BATCH 하이퍼 파라미터 16 → 64로 변경
- 상자그림과 관련된 두 라인은 커멘트아웃
- 평균제곱오차 : 0.13 / 총 학습시간 : 5.4
- DNN 파라미터 수 : 204,000개
- DNN 구조가 확장된 결과 확인
- 은닉층 3, 4번 추가
- DNN 파라미터 수 : 약 1,000,000개
- 평균제곱오차 : 0.10 / 총 학습시간 : 18
- 산포도 결과 상관관계도가 개선되었음

<정답>

- Parameter 수: 204K에서 955K로 약 4배 증가
- 손실 값: MSE는 0.13에서 0.10으로 개선
- 산포도: 추측값과 실제값의 상관관계도 개선
- 학습 시간: 5.4초에서 18.0초로 증가

<추가 문제 >

- 원래 DNN에서 은닉층 2를 삭제해 보세요.

2) 실습문제 4/7

Z-점수 정규화를 건너 뛰고 실험을 반복하세요.

결과가 어떻게 바뀌나요?

2) 실습문제 4/7

(1) 문제 분석

- Z-점수 정규화는 기계학습에 사용된 13개의 모든 요소들을 Z-점수화함
- 이를 통해 13개 요소 간의 영향력이 비슷해 짐
- 이를 건너뛰었을 때, 숫자가 큰 요소들의 중요도가 커짐 → 이는 바람직하지 않음

(2) 예상 답

- 정확도: MSE 증가 예상
- 학습 시간: 영향 없음 예상

2) 실습문제 4/7

```
187 # 케라스 DNN 구현
188 model = Sequential()
189 input_dim = X_train.shape[1]
190 model.add(Dense(288,
191                 input_dim=input_dim,
192                 activation='relu'))
193
194 model.add(Dense(1000,
195                 activation='relu'))
196
197 model.add(Dense(500,
198                 activation='relu'))
199
200 model.add(Dense(500,
201                 activation='relu'))
202
203 model.add(Dense(1))
204
205 print('\nDNN 요약')
```

- ◆ 실습문제 4
 - 은닉층 3, 4번 삭제 후 코딩 초기화
 - 평균제곱오차 : 0.11 / 총 학습시간 : 6
 - Z-점수 정규화하지 않은 경우
 - 평균제곱오차 : NaN
 - Z-점수 정규화하지 않으면 기계학습이 어려움

<정답>

- Dataset의 각 요소들의 평균과 표준편차가 각각 0과 1이 아님
- 정확도: MSE 결과는 NaN (계산 불가)
- 학습 시간: 영향 없음

<추가 문제>

- 평가용 데이터 정규화를 생략해 보세요.

2) 실습문제 5/7

학습용 데이터의 하반부를 제거하고 실험을 반복하세요.

결과가 어떻게 바뀌나요?

2) 실습문제 5/7

(1) 문제 분석

- 기계학습에 있어 데이터의 양과 질이 기계학습의 성패를 크게 좌우함

(2) 예상 답

- 정확도: MSE 증가 예상
- 학습 시간: 감소 예상

2) 실습문제 5/7

```

14 raw = pd.read_csv('housing.csv')
15
16
17 # 데이터 읽는 출력
18 print('원본 데이터 행과 열의 수')
19 print(raw.shape)
20
21
22 print('원본 데이터 통계')
23 print(raw.describe())
24
25
26 # Z-점수 정규화
27 # pandas의 n-차원 행렬 형식
28 scaler = StandardScaler()
29 X_data = scaler.fit_transform(raw)
30 Z_data = raw
31
32 # pandas의 pandas로 변환
33 # header: 행의 번호, 열의 이름
34 Z_data = pd.DataFrame(Z_data,
35                        columns=raw.columns)
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99

```

DNN 학습 최적화

[8 rows x 13 columns]

분리된 데이터 모양: (500, 13)

학습용 입력 데이터 모양: (354, 12)

학습용 출력 데이터 모양: (354,)

평가용 입력 데이터 모양: (152, 12)

평가용 출력 데이터 모양: (152,)

2020-09-26 11:56:25.744651: I tensorflow/core/platform/cpu_f...

DNN 요약

Model: "sequential_1"

Layer (type)	Output Shape	Param
dense_1 (Dense)	(None, 200)	2600
dense_2 (Dense)	(None, 1000)	20100
dense_3 (Dense)	(None, 1)	1001

Total params: 23701

실습문제 5

◆ 실습문제 5

- Z-점수 정규화 세팅 (원상복구)
- 평균제곱오차: 0.10 / 총 학습시간: 5.6
- X-train 삭제를 위해 drop 함수 사용
- Y-train 삭제를 위해 drop 함수 사용
- 평균제곱오차: 0.14 / 총 학습시간: 3.9

<정답>

- X-train 모양: (177, 12), Y-train 모양: (177,)
- 손실 값: MSE 0.10에서 0.14로 나빠짐
- 학습 시간: 5.6초에서 3.9초로 감소

<추가 문제>

- 새로운 학습용 데이터를 직접 만들어서 추가해 보세요. -> 생략

2) 실습문제 6/7

Dataset에서 CRIM (범죄율) 요소를 제거하고 실험을

반복하세요.

결과가 어떻게 바뀌나요?

2) 실습문제 6/7

(1) 문제 분석

- Dataset에서 어떤 요소는 기계학습에 도움이 되지만 어떤 요소는 방해가 됨
- 3차 수업에서 범죄율이 집값과 반비례하는 경향을 산포도에서 확인함
- 결국 범죄율 데이터가 없어지면 손실 값과 학습 시간 모두 영향을 받음
- Pandas에 drop 함수를 사용

(2) 예상 답

- 정확도: MSE 증가 예상
- 학습 시간: 약간 감소 예상

2) 실습문제 6/7

```

62 print(Z_data.describe())
63
64 # 데이터를 입력과 출력으로 분리
65 print('\n분리 된 데이터 모양:', Z_data.shape)
66 X_data = Z_data.drop('CRIM', axis=1)
67 Y_data = Z_data['MEDV']
68
69 # 데이터를 학습용과 평가용으로 분리
70 X_train, X_test, Y_train, Y_test = \
71     train_test_split(X_data,
72                     Y_data,
73                     test_size=0.3)
74
75 X_train = X_train.drop(X_train.index[177:])
76 Y_train = Y_train.drop(Y_train.index[177:])
77
78 print('\n학습용 입력 데이터 모양:', X_train.shape)
79 print('\n학습용 출력 데이터 모양:', Y_train.shape)
80 print('\n평가용 입력 데이터 모양:', X_test.shape)
81 print('\n평가용 출력 데이터 모양:', Y_test.shape)

```

DNN 학습 최적화

Model: "sequential_1"

Layer (type)	Output Shape	Param
dense_1 (Dense)	(None, 200)	2000
dense_2 (Dense)	(None, 1000)	20100
dense_3 (Dense)	(None, 1)	1001

Total params: 204,001
Trainable params: 204,001
Non-trainable params: 0

DNN 학습 시작
총 학습 시간: 6.7초
DNN 평균 제곱 오차 (MSE): 0.16

실습문제 6

◆ 실습문제 6

- drop 함수 삭제 (원상복구)
- 평균제곱오차: 0.09
- 원본 데이터의 컬럼 중 CRIM 삭제
- 평균제곱오차: 0.16
- 산포도 상관관계도 악영향

<정답>

- X-train 모양: (354, 11), X-test 모양: (152, 11)
- 손실 값: MSE가 0.09에서 0.16으로 증가
- 학습 시간: 영향 미미

<추가 문제>

- ZN을 추가 삭제해 보세요. -> 생략

2) 실습문제 7/7

집값을 예측하는 것에서, 집의 나이를 추측하는 것으로
코딩을 바꾸어 보세요.

어떤 문제가 난이도가 높은가요?

2) 실습문제 7/7

(1) 문제 분석

- Age (집 나이) 상자그림을 보면 다른 요소와 비교해 특이점도 보이지 않고 무난함
- “집값 대 집의 나이” 산포도에서 아무런 상관관계를 보이지 않음 (3차 수업 참조)
- 집값이 집의 나이보다 동네 환경에 영향을 더 많이 받는다는 것이 상식

(2) 예상 답

- 손실 값: 나이 추측이 집값 추측보다 난이도가 높을 것
- 학습 시간: 영향 없음

2) 실습문제 7/7

DNN 학습 최적화

```

22 MY_BATCH = 64
23
24
25 ##### 데이터 준비 #####
26
27 # 데이터 파일 읽기
28 # 결과는 pandas의 데이터 프레임 형식
29 heading = ['CRIM', 'ZN', 'INDUS', 'CHAS', 'NOX', 'RM',
30            'AGE', 'DIS', 'RAD', 'TAX', 'PTRATIO',
31            'LSTAT', 'MEDV']
32
33 raw = pd.read_csv('housing.csv')
34 raw = raw.drop('CRIM', axis=1)
35 heading.pop(0)
36
37
38 # 데이터 쪼개기
39 print('훈련 데이터 샘플 10개')
40 print(raw.head(10))
41
42 print('훈련 데이터 통계')
43 print(raw.describe())
44
45

```

Model: "sequential_1"

Layer (type)	Output Shape	Param
dense_1 (Dense)	(None, 200)	2600
dense_2 (Dense)	(None, 1000)	20100
dense_3 (Dense)	(None, 1)	1001

Total params: 284,681
Trainable params: 284,681
Non-trainable params: 0

DNN 학습 시작
총 학습 시간: 7.2초
DNN 평균 제공 오차 (MSE): 0.11

실습문제 7

◆ 실습문제 7

- CRIM 컬럼 복원 (원상 복구)
- 평균제곱오차 : 0.11 / 총 학습시간 : 7.2
- 집 값을 예측에서 집 나이 추측으로 바꿈
- 입력값 : 집 나이 삭제 / 출력값 : age 컬럼 사용
- 평균제곱오차 현재 : 0.25 / 총 학습시간 6.3
- 산포도 상관관계도 악영향
- 집값을 예측하는 것보다 집의 나이를 예측하는 문제가 훨씬 더 어려움

<정답>

- 손실 값: MSE가 0.11에서 0.25로 증가
-> 집의 나이 추측이 난이도가 높은 것 확인
- 학습 시간: 7.2초에서 6.3초로 감소

<추가 문제>

- CHAS (찰스강변 유무)를 대신 예측해 보세요.